

Experiment Report

Name: 李崇楷 Student ID: 313834006

Code: <https://github.com/kailee0422/Deep-Learning.git>

Introduction

In this report, we describe the design and implementation of a custom CSI_data class and its integration with PyTorch's DataLoader. The focus is on filtering data based on root name of file 、 various requirements and exporting the results to JSON files for different dataset splits (train, val, test).

Materials and Methods

This experiment utilizes the CSI_data dataset, which is divided into train, val, and test splits. The objective of this experiment is to output results based on five different requirements. To achieve this, several key functions were designed to process and filter the data.

First, the data is loaded through the `__init__(self, split='train', requirement=1)` function. The split parameter is used to specify which dataset split to use, with possible values being train, val, or test. The requirement parameter defines the criteria for filtering the data, with five possible requirements in total. Next, during the data preprocessing stage, the `filter_data(self, data_split, requirement)` function is used to filter and format the data. This function shares the same parameters as the `__init__` function. The content of the CSI_data.json file is first split into train, val, and test datasets. Each split contains different file paths, with the format `CLASS_NAME/npz/THE_GENDER_AND_COUNT/POSITION/TIME/random_characters`.

- **CLASS_NAME:** In the train set, this includes Env0 to Env5, while in the val and test sets, it includes val_set and test_set, respectively.
- **THE_GENDER_AND_COUNT and POSITION:** The number of each class has already been calculated through the last code segmentation, and can be viewed in the code's output.
- **TIME:** Composed of a series of numbers, formatted as YYMMDD_HHMMSS.

The implementation of the five requirements is as follows:

- **Requirement 1:** Search within the train, val, and test datasets to check if CLASS_NAME contains Env3.
- **Requirement 2:** Search within THE_GENDER_AND_COUNT to find classes that include two F (female) entries. Initially, a regular expression `re.match(r'F\dM\d*F\d', THE_GENDER_AND_COUNT)` was used, but during analysis, it was found that some classes like "F2M1M3F3" exist. Therefore, a simpler and more accurate method was

employed to identify entries containing two females by counting the number of latter 'F', with no restrictions on the number of males.

- Requirement 3: Use the regular expression `re.match(r'Female?', THE_GENDER_AND_COUNT)` to find classes that include one female and no males.
- Requirement 4: Directly use the conditional statement `(start_time <= TIME <= end_time)` to check if the time falls within the range from 5/6 18:13:07 to 5/7 23:24:34.
- Requirement 5: Combine the following four conditions: Search within the train, val, and test datasets to check if CLASS_NAME contains Env3. Use the regular expression `re.match(r'Male?', THE_GENDER_AND_COUNT)` to find classes that include only one male. Search within POSITION to find entries that contain 5_posi. Use the conditional statement `(start_time <= TIME <= end_time)` to check if the time falls within the range from 5/8 09:00 to 5/8 11:00.

Other functions include `__len__(self)`, which returns the size of the dataset, and `__getitem__(self, index)`, which returns samples one by one.

Finally, the code applies each requirement to the train, val, and test datasets, and writes the results to files named `A1_313834006_李崇楷_{req}.json`. Additionally, the code outputs the number of entries that match each requirement within the train, val, and test datasets. If no data matches a requirement, it outputs "{split} set, Requirement {requirement}: No matching data found." When saving the results, the message "Saved combined output for Requirement {requirement} to `./A1_313834006_李崇楷_{requirement}.json`" is displayed.

To ensure accurate classification of data, a code segment was provided for verifying the classes, but it is not directly related to this experiment and can be disregarded.

Results

```
Filtered data count for requirement 1: 158110
Filtered data count for requirement 1: 0
Val set, Requirement 1: No matching data found.
Filtered data count for requirement 1: 0
Test set, Requirement 1: No matching data found.
Saved combined output for Requirement 1 to ./A1_313834006_李崇楷_1.json
Filtered data count for requirement 2: 128820
Filtered data count for requirement 2: 0
Val set, Requirement 2: No matching data found.
Filtered data count for requirement 2: 0
Test set, Requirement 2: No matching data found.
Saved combined output for Requirement 2 to ./A1_313834006_李崇楷_2.json
Filtered data count for requirement 3: 229679
Filtered data count for requirement 3: 3622
Filtered data count for requirement 3: 4807
Saved combined output for Requirement 3 to ./A1_313834006_李崇楷_3.json
Filtered data count for requirement 4: 391067
Filtered data count for requirement 4: 0
Val set, Requirement 4: No matching data found.
Filtered data count for requirement 4: 0
Test set, Requirement 4: No matching data found.
Saved combined output for Requirement 4 to ./A1_313834006_李崇楷_4.json
Filtered data count for requirement 5: 18481
Filtered data count for requirement 5: 0
Val set, Requirement 5: No matching data found.
Filtered data count for requirement 5: 0
Test set, Requirement 5: No matching data found.
Saved combined output for Requirement 5 to ./A1_313834006_李崇楷_5.json
```

You can view its content from the above JSON file(in zip), and its format is the same as the CSI_data dataset.